

```
!pip install pandas numpy scikit-learn matplotlib seaborn wordcloud transformers torch openpyxl Sastrawi -q
```

209.7/209.7 kB 6.9 MB/s eta 0:00:00

2. Imports & Integrasi Google Drive

```
import os, re, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import torch
from torch.utils.data import Dataset

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    classification_report,
    confusion_matrix
)
from sklearn.utils.class_weight import compute_class_weight

from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    Trainer,
    TrainingArguments
)

from wordcloud import WordCloud

try:
    from google.colab import drive
    drive.mount('/content/drive')
    IS_COLAB = True
except:
    IS_COLAB = False

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
random.seed(RANDOM_STATE)
torch.manual_seed(RANDOM_STATE)

sns.set(style='whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)
```

Mounted at /content/drive

```
# Ganti path ini sesuai lokasi file Anda
FILE_PATH = '/content/drive/MyDrive/Data/dataset_gabungan_dedup.csv'

try:
    df = pd.read_csv(FILE_PATH)
    print(f"Data berhasil dimuat: {df.shape[0]} baris, {df.shape[1]} kolom")
    print(df.head(3))
except FileNotFoundError:
    print("File tidak ditemukan! Membuat dummy data untuk demonstrasi...")
    data = {
        'full_text': [
            "Program MBG ini sangat membantu anak-anak sekolah!",
            "Makan siangnya basi, parah banget pengelolaannya.",
            "Biasa aja sih, menunya standar.",
            "Semoga program ini lanjut terus, terima kasih pak presiden.",
            "Anggaran besar tapi kualitas makanan nol besar! kecewa."
        ]
    }
    df = pd.DataFrame(data)
    print("Dummy data dibuat.")
```

```
Data berhasil dimuat: 20133 baris, 19 kolom
conversation_id_str      created_at  favorite_count \
0      2.011633e+18  Thu Jan 15 02:56:06 +0000 2026      0.0
1      2.011633e+18  Thu Jan 15 02:56:03 +0000 2026      0.0
2      2.011633e+18  Thu Jan 15 02:56:02 +0000 2026      0.0

full_text      id_str \
```

```

0          Perasaan udah ga makan pedes  2.011633e+18
1 Halo #SahabatArsip Kringg Makan Nasi Pakai Ika... 2.011633e+18
2 Maksud u makan yang gak jorok kayak: 1) grille... 2.011633e+18

          image_url in_reply_to_screen_name \
0          NaN          NaN
1 https://pbs.twimg.com/amplify\_video\_thumb/2011...          NaN
2          NaN          NaN

lang location quote_count reply_count retweet_count tweet_url \
0 in NaN 0.0 0.0 0.0 NaN
1 in NaN 0.0 0.0 0.0 NaN
2 in NaN 0.0 0.0 0.0 NaN

user_id_str username source_file \
0 1.999255e+18 NaN MBG_test (1).csv
1 1.263329e+18 NaN MBG_test (1).csv
2 8.307040e+07 NaN MBG_test (1).csv

age;"job";"marital";"education";"default";"balance";"housing";"loan";"contact";"day";"month";"duration";"campaign";"pdays";"pr
0          NaN
1          NaN
2          NaN

tweet_id_clean fallback_key
0 2.011633e+18 Perasaan udah ga makan pedes | | Thu Jan 15 0...
1 2.011633e+18 Halo #SahabatArsip Kringg Makan Nasi Pakai Ika...
2 2.011633e+18 Maksud u makan yang gak jorok kayak: 1) grille...

```

```

# =====
# 1. CEK DATA AWAL
# =====

```

```

print("Jumlah data awal:", len(df))
print("Kolom dataset:", df.columns)

```

```
df.head()
```

Jumlah data awal: 20133

Kolom dataset: Index(['conversation_id_str', 'created_at', 'favorite_count', 'full_text', 'id_str', 'image_url', 'in_reply_to_screen_name', 'lang', 'location', 'quote_count', 'reply_count', 'retweet_count', 'tweet_url', 'user_id_str', 'username', 'source_file', 'age', 'job', 'marital', 'education', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'tweet_id_clean', 'fallback_key'], dtype='object')

	conversation_id_str	created_at	favorite_count	full_text	id_str	image_url	in_re
0	2.011633e+18	Thu Jan 15 02:56:06 +0000 2026	0.0	Perasaan udah ga makan pedes	2.011633e+18	NaN	
1	2.011633e+18	Thu Jan 15 02:56:03 +0000 2026	0.0	Halo #SahabatArsip Kringg Makan Nasi Pakai Ika...	2.011633e+18	https://pbs.twimg.com/amplify_video_thumb/2011...	
2	2.011633e+18	Thu Jan 15 02:56:02 +0000 2026	0.0	Maksud u makan yang gak jorok kayak: 1) grille...	2.011633e+18	NaN	
3	2.011633e+18	Thu Jan 15 02:55:59 +0000 2026	0.0	Makan mie enak kali ya	2.011633e+18	NaN	
4	2.011479e+18	Thu Jan 15 02:55:55 +0000 2026	0.0	@symplecci Jadi pengen makan apel	2.011633e+18	NaN	

Langkah berikutnya:

[Buat kode dengan df](#)

[New interactive sheet](#)

```

# =====
# 2. SELEKSI KOLOM TEKS

```

```
# =====
```

```
df_clean = df.copy()
```

```
df_clean = df_clean[['full_text']].dropna()
```

```
df_clean = df_clean.drop_duplicates(subset=['full_text']).reset_index(drop=True)
```

```
print("Jumlah data setelah hapus kosong dan duplikat:", len(df_clean))
```

```
df_clean.head()
```

Jumlah data setelah hapus kosong dan duplikat: 19919

	full_text	
0	Perasaan udah ga makan pedes	
1	Halo #SahabatArsip Kringg Makan Nasi Pakai Ika...	
2	Maksud u makan yang gak jorok kayak: 1) grille...	
3	Makan mie enak kali ya	
4	@symplecci Jadi pengen makan apel	

Langkah berikutnya:

[Buat kode dengan df_clean](#)

[New interactive sheet](#)

```
# =====
```

```
# 3. PREPROCESSING UNTUK INDOBERTWEET
```

```
# =====
```

```
norm_dict = {
```

```
    'bgt': 'banget',
```

```
    'ga': 'tidak',
```

```
    'gak': 'tidak',
```

```
    'nggak': 'tidak',
```

```
    'g': 'tidak',
```

```
    'tdk': 'tidak',
```

```
    'yg': 'yang',
```

```
    'dlu': 'dulu',
```

```
    'kalo': 'kalau',
```

```
    'klo': 'kalau',
```

```
    'knp': 'kenapa',
```

```
    'bkn': 'bukan',
```

```
    'udh': 'sudah',
```

```
    'sdh': 'sudah',
```

```
    'dah': 'sudah',
```

```
    'jgn': 'jangan',
```

```
    'aja': 'saja',
```

```
    'tpi': 'tapi',
```

```
    'tp': 'tapi',
```

```
    'krn': 'karena',
```

```
    'utk': 'untuk',
```

```
    'u': 'aku',
```

```
    'dpt': 'dapat',
```

```
    'lg': 'lagi',
```

```
    'trs': 'terus',
```

```
    'kek': 'kayak',
```

```
    'gw': 'aku',
```

```
    'gue': 'aku',
```

```
    'gua': 'aku',
```

```
    'pd': 'pada',
```

```
    'mbg': 'makan bergizi gratis'
```

```
}
```

```
def normalize_text(text):
```

```
    words = str(text).split()
```

```
    words = [norm_dict.get(word, word) for word in words]
```

```
    return " ".join(words)
```

```
def clean_text_indobertweet(text):
```

```
    if not isinstance(text, str):
```

```
        return ""
```

```
    text = text.lower()
```

```
    text = re.sub(r'http\S+|www.\S+|https\S+', '', text)
```

```
    text = re.sub(r'@\w+', '@USER', text)
```

```
    # hashtag: simbol # dihapus, kata tetap dipertahankan
```

```
    text = re.sub(r'#', '', text)
```

```
    # karakter HTML umum
```

```

text = re.sub(r'&','dan', text)

# hapus karakter aneh berulang, tetapi jangan terlalu agresif
text = re.sub(r'\s+', ' ', text).strip()

text = normalize_text(text)

return text

df_clean['text_clean'] = df_clean['full_text'].apply(clean_text_indobertweet)

print("Preprocessing IndoBERTweet selesai.")
df_clean[['full_text', 'text_clean']].head()

```

Preprocessing IndoBERTweet selesai.

	full_text	text_clean
0	Perasaan udah ga makan pedes	perasaan udah tidak makan pedes
1	Halo #SahabatArsip Kringg Makan Nasi Pakai Ika...	halo sahabatarsip kringg makan nasi pakai ikan...
2	Maksud u makan yang gak jorok kayak: 1) grille...	maksud aku makan yang tidak jorok kayak: 1) gr...
3	Makan mie enak kali ya	makan mie enak kali ya
4	@symplecci Jadi pengen makan apel	@USER jadi pengen makan apel

```

# =====
# 4. CEK HASIL PREPROCESSING
# =====

print("Jumlah data setelah preprocessing:", len(df_clean))
print("Jumlah teks kosong:", (df_clean['text_clean'].str.strip() == "").sum())

df_clean[['full_text', 'text_clean']].sample(10, random_state=42)

```

Jumlah data setelah preprocessing: 19919
Jumlah teks kosong: 67

	full_text	text_clean
18205	@brutalouttahere Boleh bingit #SobatRupiah! Pe...	@USER boleh bingit sobatrupiah! permohonan mag...
6950	cn #China *#PBOC TO BOOST RELENDING QUOTA FO...	cn china *pboc to boost relending quota for ...
4265	MBG harganya anggaran pendidikan dan kesehatan...	makan bergizi gratis harganya anggaran pendidi...
10108	@apoorv_bh89 @WeaveGrid @SCE @Tesla I didn't g...	@USER @USER @USER @USER i didn't get it at fir...
856	Dari awal masuk KKN sampe hari ini gua belum m...	dari awal masuk kkn sampe hari ini aku belum m...
7051	@CryBabyAubies unc fans been screaming to get ...	@USER unc fans been screaming to get hubert ou...
3616	@RichSilver I want to agree If you can convinc...	@USER i want to agree if you can convince me i...
14441	America kill their own children with tear gas ...	america kill their own children with tear gas ...
7221	@CaptainBrady3 @ohmike83 @PhthalateSoup @NAZIt...	@USER @USER @USER @USER @USER @USER @USER base...
2748	kalau gw jadi dpr program gw 1 rumah 1 woojin ...	kalau aku jadi dpr program aku 1 rumah 1 woojin

```

df_clean = df_clean[df_clean['text_clean'].str.strip() != ""].reset_index(drop=True)

print("Jumlah data final setelah menghapus teks kosong:", len(df_clean))

```

Jumlah data final setelah menghapus teks kosong: 19852

```
print(df_clean.columns)
```

```
Index(['full_text', 'text_clean'], dtype='object')
```

```

MODEL_NAME = "Aardiiiyy/indobertweet-base-Indonesian-sentiment-analysis"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
model.eval()

print("Device:", device)

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100% 994/994 [00:00<00:00, 13.0kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
tokenizer_config.json: 1.26k/? [00:00<00:00, 12.9kB/s]

vocab.txt: 235k/? [00:00<00:00, 3.12MB/s]

special_tokens_map.json: 100% 695/695 [00:00<00:00, 12.7kB/s]

model.safetensors: 100% 442M/442M [00:08<00:00, 91.4MB/s]

Loading weights: 100% 201/201 [00:00<00:00, 339.58it/s, Materializing param=classifier.weight]
Device: cpu

```

```

from tqdm import tqdm

texts = df_clean['text_clean'].astype(str).tolist()

batch_size = 32
labels = []
confidences = []

for i in tqdm(range(0, len(texts), batch_size)):
    batch_texts = texts[i:i+batch_size]

    inputs = tokenizer(
        batch_texts,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=128
    ).to(device)

    with torch.no_grad():
        outputs = model(**inputs)
        probs = torch.softmax(outputs.logits, dim=1)

    preds = torch.argmax(probs, dim=1).cpu().numpy()
    confs = torch.max(probs, dim=1).values.cpu().numpy()

    for p, c in zip(preds, confs):
        if p == 0:
            labels.append("negatif")
        elif p == 1:
            labels.append("netral")
        else:
            labels.append("positif")

        confidences.append(c)

df_clean['label'] = labels
df_clean['confidence'] = confidences

100%|██████████| 621/621 [1:45:32<00:00, 10.20s/it]

```

```
df_clean['label'].value_counts()
```

	count
label	
positif	8625
netral	6721
negatif	4506

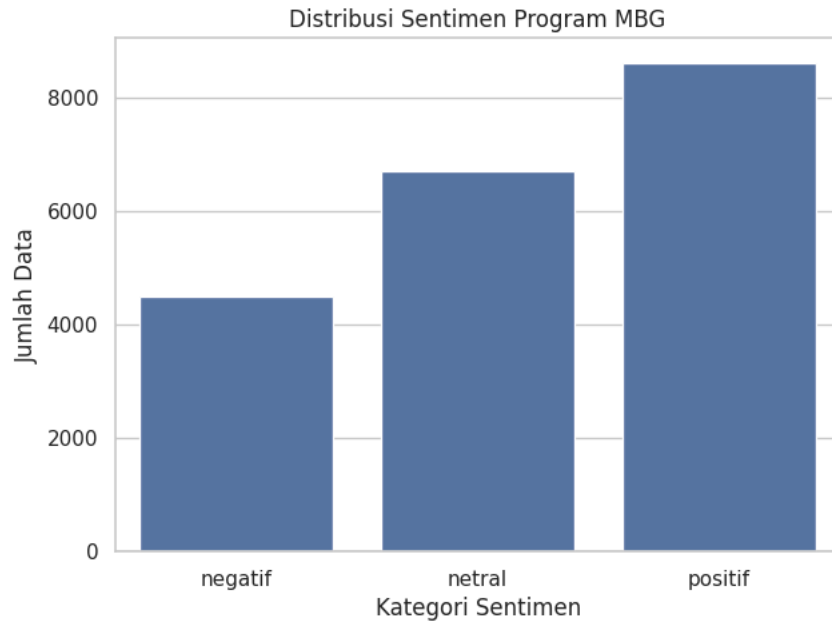
dtype: int64

```

import seaborn as sns
import matplotlib.pyplot as plt

```

```
plt.figure(figsize=(7,5))
sns.countplot(
    data=df_clean,
    x='label',
    order=['negatif', 'netral', 'positif']
)
plt.title("Distribusi Sentimen Program MBG")
plt.xlabel("Kategori Sentimen")
plt.ylabel("Jumlah Data")
plt.show()
```



```
df_clean.to_csv("hasil_labeling_mbg_indobertweet.csv", index=False)
```

```
#encode label menjadi angka
label_map = {
    "negatif": 0,
    "netral": 1,
    "positif": 2
}

df_clean['label_id'] = df_clean['label'].map(label_map)

df_clean[['text_clean', 'label', 'label_id']].head()
```

	text_clean	label	label_id
0	perasaan udah tidak makan pedes	negatif	0
1	halo sahabatarsip kringg makan nasi pakai ikan...	netral	1
2	maksud aku makan yang tidak jorok kayak: 1) gr...	netral	1
3	makan mie enak kali ya	netral	1
4	@USER jadi pengen makan apel	positif	2

```
#split data 70:15:15
from sklearn.model_selection import train_test_split

train_texts, temp_texts, train_labels, temp_labels = train_test_split(
    df_clean['text_clean'].tolist(),
    df_clean['label_id'].tolist(),
    test_size=0.30,
    random_state=42,
    stratify=df_clean['label_id']
)

val_texts, test_texts, val_labels, test_labels = train_test_split(
    temp_texts,
    temp_labels,
    test_size=0.50,
```

```

        random_state=42,
        stratify=temp_labels
    )

    print("Data latih:", len(train_texts))
    print("Data validasi:", len(val_texts))
    print("Data uji:", len(test_texts))

```

```

Data latih: 13896
Data validasi: 2978
Data uji: 2978

```

```

#tokenisasi untuk training
train_encodings = tokenizer(
    train_texts,
    truncation=True,
    padding=True,
    max_length=128
)

val_encodings = tokenizer(
    val_texts,
    truncation=True,
    padding=True,
    max_length=128
)

test_encodings = tokenizer(
    test_texts,
    truncation=True,
    padding=True,
    max_length=128
)

```

```

#buat dataset Pytorch
import torch
from torch.utils.data import Dataset

class SentimentDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {
            key: torch.tensor(val[idx])
            for key, val in self.encodings.items()
        }
        item['labels'] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)

train_dataset = SentimentDataset(train_encodings, train_labels)
val_dataset = SentimentDataset(val_encodings, val_labels)
test_dataset = SentimentDataset(test_encodings, test_labels)

```

```

#load model untuk klasifikasi
from transformers import AutoModelForSequenceClassification

MODEL_NAME = "Aardiii/indobertweet-base-Indonesian-sentiment-analysis"

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=3
)

```

Loading weights: 100%

201/201 [00:00<00:00, 492.55it/s, Materializing param=classifier.weight]

```

#training /fine-tuning
from transformers import Trainer, TrainingArguments
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import numpy as np

def compute_metrics(pred):
    labels = pred.label_ids
    preds = np.argmax(pred.predictions, axis=1)

```

```

precision, recall, f1, _ = precision_recall_fscore_support(
    labels,
    preds,
    average='weighted',
    zero_division=0
)

acc = accuracy_score(labels, preds)

return {
    'accuracy': acc,
    'precision': precision,
    'recall': recall,
    'f1': f1
}

training_args = TrainingArguments(
    output_dir='./hasil_indobertweet_mbg',
    eval_strategy='epoch',
    save_strategy='epoch',
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=4,
    weight_decay=0.01,
    logging_dir='./logs',
    load_best_model_at_end=True,
    metric_for_best_model='f1'
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics
)

trainer.train()

```

```

#evaluasi pada data uji
predictions = trainer.predict(test_dataset)

y_pred = np.argmax(predictions.predictions, axis=1)
y_true = np.array(test_labels)

```

```

#clasification Repist
from sklearn.metrics import classification_report, accuracy_score

target_names = ["negatif", "netral", "positif"]

print("Accuracy:", accuracy_score(y_true, y_pred))

print(classification_report(
    y_true,
    y_pred,
    target_names=target_names,
    digits=4
))

```



```

#confusion Matrix
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=target_names,
    yticklabels=target_names
)

plt.title("Confusion Matrix IndoBERTweet")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

```

```

#simpan model
trainer.save_model("./model_indobertweet_mbg")
tokenizer.save_pretrained("./model_indobertweet_mbg")

```

```

#word cloud per sentimen
from wordcloud import WordCloud

for label in ["negatif", "netral", "positif"]:
    text = " ".join(df_clean[df_clean["label"] == label]["text_clean"])

    wc = WordCloud(
        width=800,
        height=400,
        background_color="white"
    ).generate(text)

    plt.figure(figsize=(10,5))
    plt.imshow(wc, interpolation="bilinear")
    plt.axis("off")
    plt.title(f"Word Cloud Sentimen {label.capitalize()}")
    plt.show()

```